

CODEARENA'26 · IDEATHON TOPICS

Choose Your Challenge

Five problems, one system shape, 36 hours to build it end to end.

IET On Campus KDU, in collaboration with IET Young Professionals Network Sri Lanka

Released 11 September 2026



Five topics, one shape

Each topic has a one-page brief and a reference flow. Pick one as a team and commit to it together.

01	TELECOM	Detecting and fixing mobile service problems at scale	Combine customer complaints with live network data to find, fix, and explain service problems.
02	FINTECH & TRUST	Defending wallet users from fraud and scams	Link scam messages, SIM swaps, and suspicious payments into one defence that stops fraud early.
03	LOGISTICS	Managing last-mile delivery problems in real time	Watch every live delivery, catch problems before customers complain, and settle disputes fairly.
04	DISASTER RESPONSE	Coordinating city response to floods and road hazards	Verify citizen reports against live weather data, warn people at risk, and dispatch crews and relief.
05	AGRICULTURE	Early warning for crop disease across a region	Diagnose crop disease from photos, advise farmers in their language, and catch outbreaks early.

Every topic follows the same end-to-end flow

Read each reference flow top to bottom. Your design can differ, but each stage must be real and working.

01	Two ways in	A user reports a problem in the app, or a live data feed (mocked by you) raises it before anyone complains.
02	Case builder	Plain backend code joins the report with everything the checks need from your database.
03	Five checks	Each check reads one signal. SYSTEM checks are plain code and rules; AI checks call a model from your backend, never the frontend.
04	Aggregator	An AI step weighs every signal into one verdict with a confidence score, and shows its reasons.
05	Four outcomes	Resolve automatically, loop back to the user, broadcast to everyone affected, or raise a ticket for a person.
06	People and loops	Tickets move through a dashboard and a four-step human chain. Final decisions feed back to tune the AI.

Suggested minimum scope: the app, a mocked live feed, three checks (at least one SYSTEM), the aggregator, and one dashboard, working end to end. Everything beyond that is extra credit.

Detecting and fixing mobile service problems at scale

Find network, billing, and device problems before customers call, and fix what you safely can.

THE PROBLEM

Customers blame the network for everything, and hotline agents often guess. Meanwhile, the network raises its own alarms that nobody connects to the complaints coming in. Build a platform that joins customer reports with live network data, finds the real cause of each problem, fixes what it can automatically, and tells every affected customer what's happening.

WHO USES IT

Customer	Reports problems in the app and receives updates
NOC engineer	Watches the live map and dispatches field teams
Field engineer	Fixes site faults and closes jobs with a report
System admin	Issues credits and tunes how strict the AI is

MUST WORK END TO END

- 01 A customer can report a problem in the app and watch its status change
- 02 A mocked network feed raises alarms that link to the complaints they explain
- 03 At least three checks run on every case, including one plain system check
- 04 An outage triggers a broadcast to every customer on the affected site
- 05 Tickets flow to a field engineer, and closing one notifies the customer

DATA YOU WILL MOCK

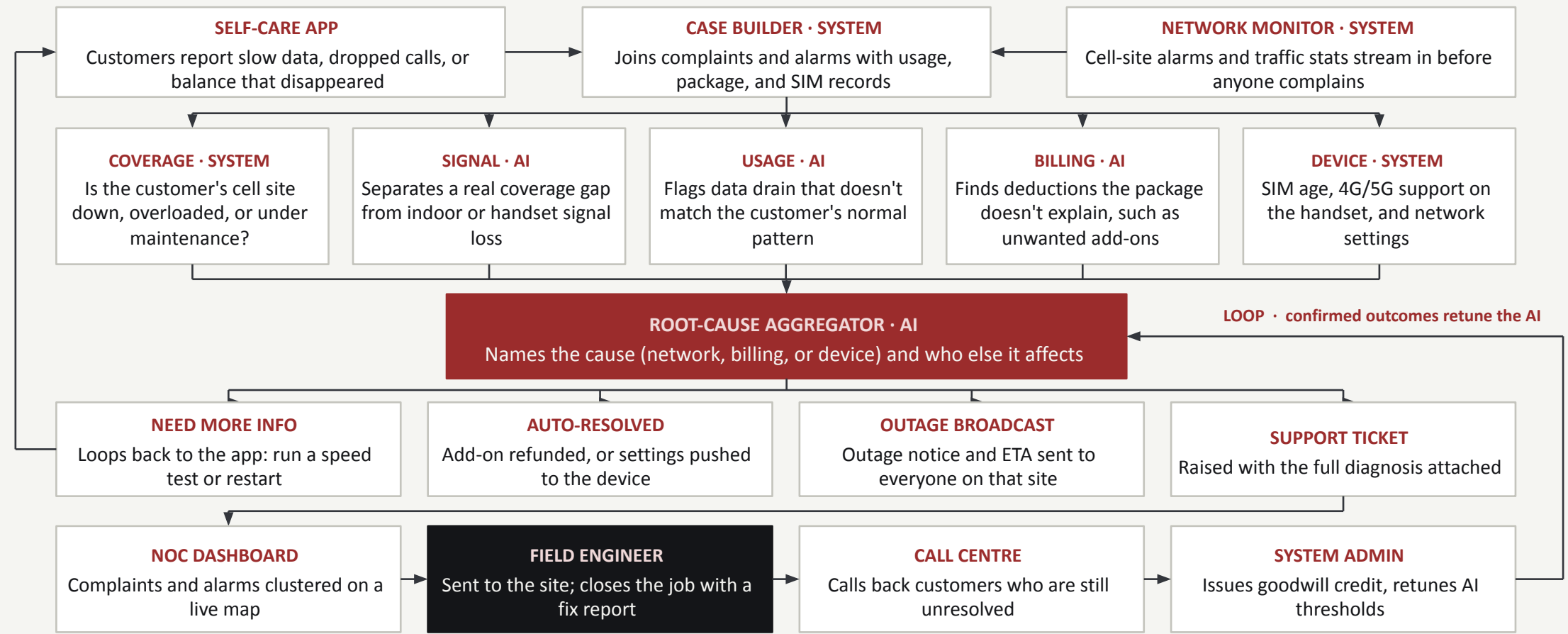
Customers, packages, usage logs, and 20 to 30 cell sites, with a script that replays alarms and outages over time.

STRETCH GOAL

Predict which sites are likely to fail next from their alarm history, before customers notice.

Detecting and fixing mobile service problems at scale

Reference design only. Your architecture can differ, but every stage shown here must work end to end.



Defending wallet users from fraud and scams

Scam messages lead to stolen OTPs, SIM swaps, and drained wallets. Break the chain at every step.

THE PROBLEM

Most wallet fraud follows the same chain: a scam message, a stolen OTP or a SIM swap, and then a drained wallet. Each link is usually handled by a different team, so the chain is only spotted after the money is gone. Build a platform that watches every link at once, stops risky payments in real time, and shuts down scam campaigns for all users.

WHO USES IT

Wallet user	Disputes payments and reports scam messages
Fraud analyst	Investigates fraud rings and confirms campaigns
Call centre agent	Verifies identity and supports victims
System admin	Reverses payments and tunes the fraud rules

MUST WORK END TO END

- 01A user can dispute a payment or forward a scam message from the app
- 02A mocked transaction stream is screened live, and risky payments are held
- 03SIM-swap and device checks run as plain code; behaviour and message checks use AI
- 04Confirming a scam campaign blocks it and warns every user
- 05An analyst's decision updates the case and notifies the user

DATA YOU WILL MOCK

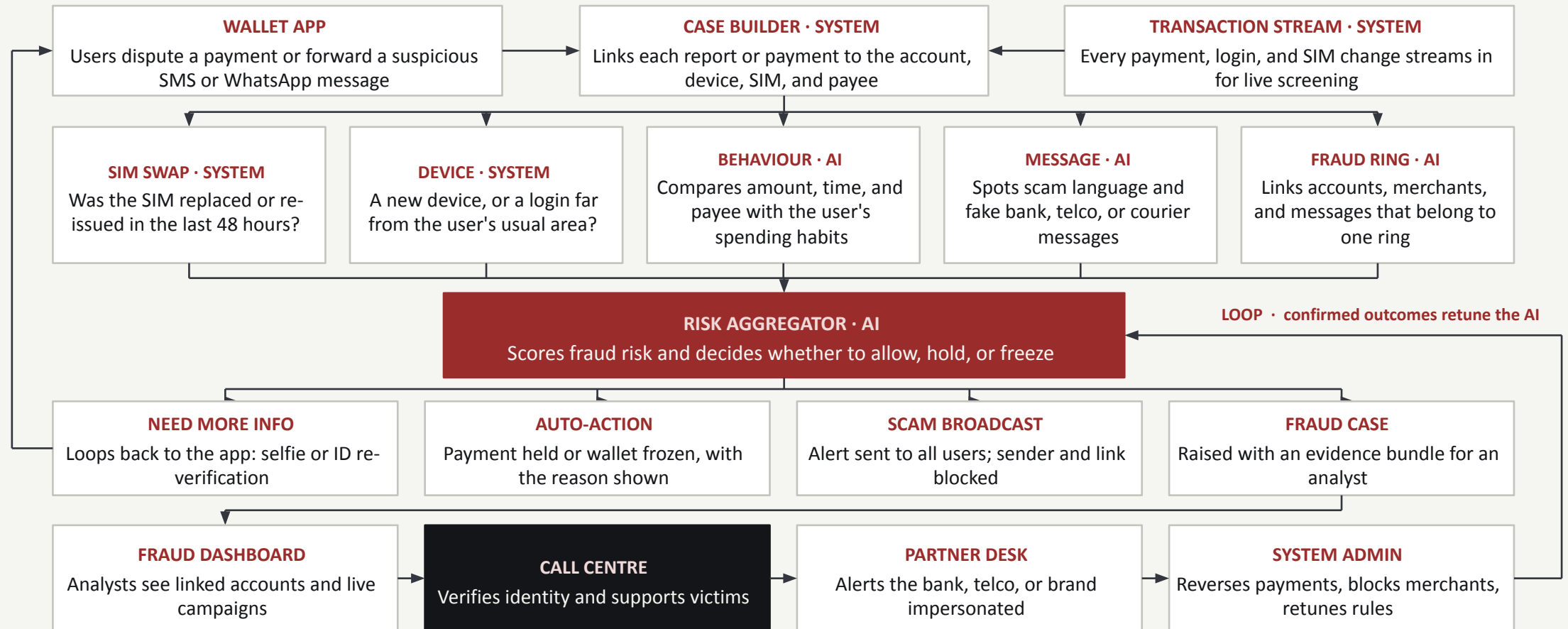
Users with months of transactions, devices, and SIM changes, plus 50 to 100 scam and genuine messages you write, with one fraud ring planted.

STRETCH GOAL

A graph view showing how one scam message led to linked accounts and payments.

Defending wallet users from fraud and scams

Reference design only. Your architecture can differ, but every stage shown here must work end to end.



Managing last-mile delivery problems in real time

Late, wrong, damaged, or undelivered: catch problems as they happen and settle them fairly.

THE PROBLEM

Delivery platforms usually find out about problems only when an angry customer complains. Then they either refund everything and lose money to abuse, or question everything and lose customers, while merchants and riders get blamed for each other's mistakes. Build a platform that watches every live delivery, catches problems early, and settles each complaint fairly.

WHO USES IT

Customer	Reports problems and receives updates
Merchant partner	Responds to complaints about their orders
Ops team	Watches live deliveries and zone-wide delays
Support agent	Mediates disputes and handles refund abuse

MUST WORK END TO END

- 01A customer can report an order problem with a photo in the app
- 02A mocked live order feed flags late orders before any complaint arrives
- 03The verdict names who is at fault (merchant, rider, or neither) with reasons
- 04A zone-wide delay triggers a notice to every affected customer
- 05Valid claims are refunded automatically; disputes reach the partner portal

DATA YOU WILL MOCK

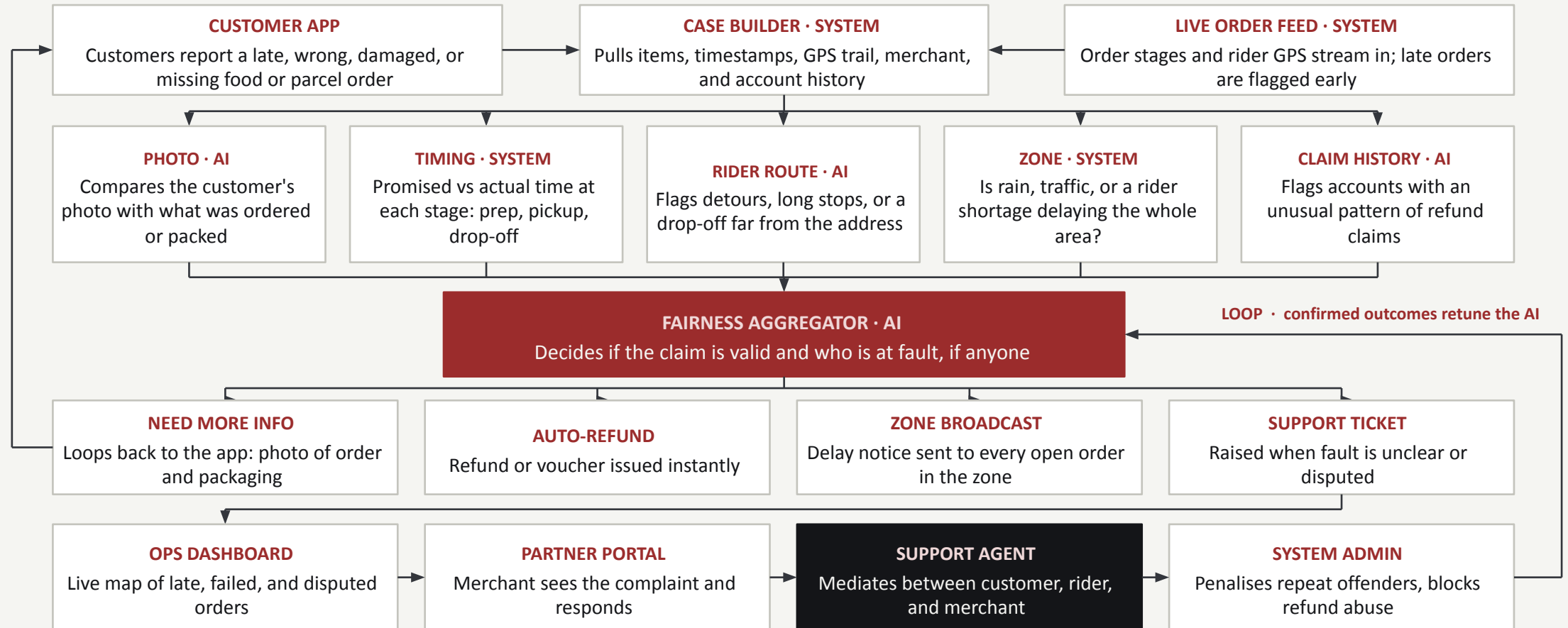
Merchants, orders with stage timestamps, rider GPS trails, and customer refund histories, plus a script that replays a busy evening.

STRETCH GOAL

Reassign orders to another rider automatically when one falls far behind.

Managing last-mile delivery problems in real time

Reference design only. Your architecture can differ, but every stage shown here must work end to end.



Coordinating city response to floods and road hazards

Verify what citizens report, warn the people at risk, and get crews and relief where they're needed.

THE PROBLEM

During heavy rain, roads flood and trees fall faster than councils can find out. Citizens report hazards and ask for help on social media, but nobody can verify the reports or tell who needs help first. Build a platform that verifies citizen reports against live weather data, keeps a trusted hazard map, warns people at risk, and sends crews and relief where they're needed.

WHO USES IT

Citizen	Reports hazards and requests help
Council officer	Reviews hazards and dispatches crews
Field crew	Fixes hazards and closes reports with a photo
Relief coordinator	Matches help requests to shelters and supplies

MUST WORK END TO END

- 01A citizen can report a hazard or request help with a photo and GPS location
- 02A mocked weather and river feed raises area warnings on its own
- 03Weather and cluster checks run as plain code; image and location checks use AI
- 04A confirmed flood triggers an alert with safe routes for everyone in the area
- 05A crew closes a hazard with a photo, which updates the public map

DATA YOU WILL MOCK

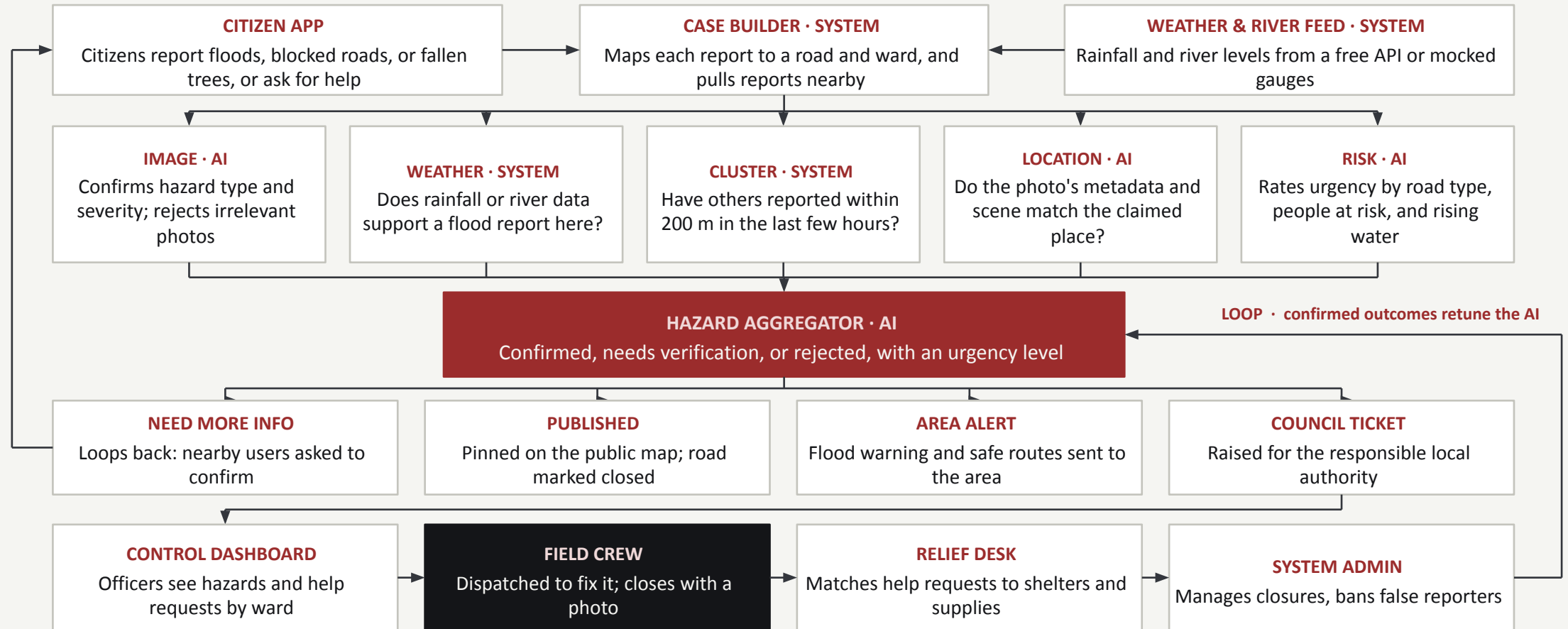
Hazard photos, reports across a few city wards, shelters with capacity, and a script that replays a past rainy day's weather data.

STRETCH GOAL

Suggest detours around closed roads and route relief to the nearest shelter with space.

Coordinating city response to floods and road hazards

Reference design only. Your architecture can differ, but every stage shown here must work end to end.



Early warning for crop disease across a region

Diagnose sick plants from a photo, advise farmers in their language, and stop outbreaks spreading.

THE PROBLEM

Farmers can wait days for an agriculture officer, and by then a disease may have spread across the field and to neighbouring farms. Officers can't see where outbreaks are starting until it's too late. Build a platform that diagnoses crop problems from photos, advises farmers in their own language, spots outbreaks as they form, and warns every farm at risk.

WHO USES IT

Farmer	Uploads photos and receives treatment advice
Agriculture officer	Reviews cases and visits farms
Research lab	Tests samples of new or unclear diseases
System admin	Declares outbreaks and improves the AI

MUST WORK END TO END

- 01A farmer can upload leaf photos and get a diagnosis in Sinhala, Tamil, or English
- 02Weather and outbreak checks run as plain code; image and severity checks use AI
- 03Low-confidence diagnoses go to an officer instead of giving the farmer a guess
- 04Reports cluster on a regional map, and a confirmed outbreak warns nearby farms
- 05An officer's field visit updates the case, and the farmer is notified

DATA YOU WILL MOCK

Plant disease images from an open dataset such as PlantVillage, farms with crops and locations, and weather data from a free API.

STRETCH GOAL

Voice input and read-aloud advice for farmers who prefer not to type.

Early warning for crop disease across a region

Reference design only. Your architecture can differ, but every stage shown here must work end to end.

